Atomic Decision Boundaries: A Structural Requirement for Guaranteeing Execution-Time Admissibility in Autonomous Systems

Marcelo Fernandez
TraslaIA
info@traslaia.com

April 2026 (Draft)

Abstract

Autonomous systems increasingly execute actions that directly modify shared state, creating an urgent need for precise control over which transitions are permitted to occur. Existing governance mechanisms evaluate policies prior to execution or reconstruct behavior post hoc, but do not enforce admissibility at the exact moment a state transition is committed.

We introduce the *atomic decision boundary*, a structural property of admission control systems in which the decision and the resulting state transition are jointly determined as a single indivisible step. Formalizing execution as a labeled transition system (LTS), we distinguish two classes: *atomic systems*, where evaluation and transition are coupled within a single LTS step, and *split evaluation systems*, where they are separate transitions that may be interleaved by environmental actions.

Under realistic concurrent environments (Assumptions 2.1–2.2), we prove that no construction can make a split system equivalent to an atomic system with respect to admissibility under all execution traces. This limitation is a structural property of the system architecture, not of policy expressiveness or state availability. We further formalize the Escalate outcome—absent from classical time-of-check/time-of-use (TOCTOU) analyses— and show that its resolution is itself subject to the atomic boundary requirement, extending the guarantees of the main theorem.

We map widely used policy enforcement mechanisms, including RBAC and Open Policy Agent (OPA), to the split model, and contrast them with systems that enforce atomic decision boundaries. Our results establish that admissibility is a property of execution, not of prior evaluation, and that guaranteeing it requires structural guarantees at the decision boundary.

1 Introduction

Autonomous systems are increasingly deployed in settings where they plan and execute multi-step action sequences that directly and irreversibly modify system state. Financial transfers, document operations, API calls, and inter-agent delegations all commit changes that cannot simply be rolled back. As a result, the question of whether a given action should be allowed to execute must be resolved precisely at the moment the transition is committed—not before, and not after.

Consider a concrete scenario. An agent is authorized to transfer funds from a shared ledger. A policy engine evaluates the request against the current balance and account status, returns Allow, and triggers the transfer. In the interval between that evaluation and the execution of the debit, a concurrent transaction—another agent repaying a loan—reduces the balance below the required

amount. The transfer executes against an inadmissible state. The guarantee the policy was meant to enforce has evaporated. No refinement of the policy expression could have prevented this: the failure is not in what the policy *knows*, but in *when* the transition fires relative to the state it governs. We call the gap between these two moments the *decision boundary problem*, and formalize it precisely in what follows.

Current governance approaches address this question indirectly. *Policy engines* evaluate rules against input data prior to execution. *Access control models* determine permissions based on static role or attribute assignments. *Audit systems* reconstruct behavior after the fact. These mechanisms are useful and widely deployed, but they share a structural limitation: the decision to allow an action and the transition that applies that action are separate operations.

This separation introduces a gap. The decision is evaluated in one system state; the transition fires in a—potentially different—state. An action admissible when evaluated may become inadmissible by execution time. Under the environment model we formalize in §2, no policy, regardless of its sophistication, can close this gap from within a split architecture: the gap is not an information problem—it is an architectural one.

This paper formalizes that observation precisely. We define the *decision boundary* of a governance system as the point at which the system resolves whether a state transition ($s \to s'$) is permitted to exist. We then define an *atomic decision boundary* as one in which the resolution and the resulting transition are jointly computed as a single indivisible step in the labeled transition system model of execution.

We prove that systems lacking atomicity at the decision boundary—henceforth *split evaluation systems*—cannot guarantee admissibility at execution time under all execution traces. The impossibility is constructive: we exhibit a concrete execution trace that witnesses the gap.

A distinguishing feature of our model is the three-valued decision domain $\mathcal{D} = \{\textsc{Allow}, \textsc{Refuse}, \textsc{Escalate}\}$. The Escalate outcome—absent from classical TOCTOU analyses [2, 14]—suspends the transition pending supervisor review, a first-class governance action in modern agent systems [4]. We show that Escalate does not eliminate the atomicity requirement; it *transfers* it to the resolution step, and prove that the same structural gap arises there unless resolution is itself atomic.

Contributions.

(i) We formalize the execution model for admission control as an LTS and define admissibility as a property of state transitions, not of actions in isolation (§2).

(ii) We define the *atomic decision boundary* in LTS terms and contrast it with *split evaluation systems* via a precise structural distinction (§3).

(iii) We give a complete formal semantics for the Escalate outcome, including the supervisor resolution function and the state it operates over (§3).

(iv) We prove, via a constructive counterexample trace, that no general construction can make a split system equivalent to an atomic system with respect to admissibility (§5). Three corollaries follow: impossibility of execution-time guarantees in split systems, insufficiency of external state, and admissibility as an execution-time property.

(v) We prove the *escalation closure requirement*: the Escalate outcome preserves admissibility guarantees if and only if the supervisor resolution is itself atomic (§5).

(vi) We map RBAC, OPA, and ACP to the two classes, providing a structural taxonomy of existing governance mechanisms (§6).

This paper is Paper 0 in a series of four. The Agent Control Protocol (ACP) [4] (Paper 1) instantiates the atomic decision boundary for multi-agent governance. The Invariant Measurement Layer (IML) [6] (Paper 2) operates above the atomic enforcement layer and addresses behavioral drift that remains invisible to enforcement-based monitoring. Paper 3 [5] addresses the fair allocation of atomic governance decisions across agents operating under shared resource constraints.

2 System Model

We model the execution environment as a labeled transition system and derive the formal definition of admissibility from it. The section is organized into two parts: an execution model (§2.1) that establishes the LTS and the admissibility predicate, and an environment and threat model (§2.2) that specifies the assumptions under which the main theorem is stated.

2.1 Execution Model

Definition 2.1 (Labeled Transition System)**.** A *labeled transition system* (LTS) is a triple $M = (S, \mathrm{Act}, \rightarrow)$ where:

- S is a set of *states*;
- $\mathrm{Act} = A \cup A_{\mathrm{env}}$ is a set of *action labels*, partitioned into *agent actions* A (subject to governance) and *environment actions* A_{env} (external state modifications), with $A \cap A_{\mathrm{env}} = \emptyset$;
- $\rightarrow \subseteq S \times \mathrm{Act} \times S$ is the *transition relation.*

We write $s \xrightarrow{a} s'$ for $(s, a, s') \in \rightarrow$.

Definition 2.2 (Execution Trace)**.** An *execution trace* σ is a finite alternating sequence $s_0\, a_1\, s_1\, a_2\, s_2 \cdots a_n\, s_n$ such that $s_{i-1} \xrightarrow{a_i} s_i$ for all $i \in \{1, \ldots, n\}$. We denote the set of all execution traces of M by $\mathrm{Traces}(M)$.

Definition 2.3 (Admissibility)**.** An *admissibility predicate* is a function $\mathrm{Adm} : S \times A \rightarrow \{\mathsf{true}, \mathsf{false}\}$. Action $a \in A$ is *admissible* in state $s \in S$ if $\mathrm{Adm}(s, a) = \mathsf{true}$. Admissibility is a property of the state at the moment of transition, not of the action in isolation.

Definition 2.4 (Admissibility-Preserving System)**.** A system M *preserves admissibility* if for every trace $\sigma \in \mathrm{Traces}(M)$ and every agent action $a \in A$ appearing in σ, whenever $s \xrightarrow{a} s'$ occurs in σ, we have $\mathrm{Adm}(s, a) = \mathsf{true}$.

The evaluation–execution gap. In many practical systems, deciding whether to allow an action and applying the resulting transition are implemented as distinct operations. Let $D : S \times A \rightarrow \mathcal{D}$ denote the *decision function* that evaluates admissibility, and let $T : S \times A \rightarrow S$ denote the *transition function* that applies it. In split systems, D and T are invoked separately. The state in which D is invoked, call it s_{eval}, need not equal the state in which T fires, call it s_{exec}. This is the structural source of the problem we formalize.

2.2 Environment and Threat Model

The main theorem (Theorem 5.1) is a conditional result: it holds under a specific model of the execution environment. We state this model explicitly here, as two assumptions, so that the scope of the result is unambiguous. The environment model is *adversarial*: the environment is assumed capable of interleaving state-modifying actions at the worst possible moment, with no synchronization guarantee for the governance system. This is the standard assumption in concurrent systems analysis and corresponds to the worst-case deployment context for multi-agent systems operating over shared mutable state. Readers who restrict to sequential or single-threaded settings should consult Remark 5.2 for the precise boundary conditions under which the theorem applies.

Assumption 2.5 (Non-Triviality)**.** The system satisfies the following:

(i) There exist $s \in S$ and $a \in A$ with $\mathrm{Adm}(s, a) = \mathsf{true}$ and $D(s, a) = \textsc{Allow}$ (the decision function grants at least one admissible action without escalation).

(ii) There exist $s \in S$, $a \in A$, and $e \in A_{\mathrm{env}}$ such that $s \xrightarrow{e} s^*$ and $\mathrm{Adm}(s^*, a) = \mathsf{false}$.

Assumption 2.5 is satisfied by any system where admissibility genuinely depends on state. A governance mechanism over a state-independent system provides no meaningful constraint; a system whose state cannot change between decision and execution is already effectively atomic. In practice, every multi-agent system operating over shared mutable state—including file systems, API quota registries, agent delegation ledgers, and shared databases— satisfies both conditions: actions can be admitted and state can be modified by concurrent environment processes between evaluation and execution.

Assumption 2.6 (Environment Model)**.** The environment satisfies the following:

(i) *Uncontrollability.* Environment actions $e \in A_{\mathrm{env}}$ are not subject to governance control: the admission system cannot prevent, delay, or reorder them.

(ii) *Arbitrary interleaving.* In a split evaluation system, environment actions may fire at any point in the LTS between the decision transition $dec(a)$ and the execution transition $exec(a)$. The system provides no synchronization barrier that would prevent such interleaving.

(iii) *State visibility.* The decision function D observes the system state at the time it is invoked. It does not have predictive access to future states produced by environment actions that have not yet fired.

Assumption 2.6(i) reflects the standard distinction between controllable and uncontrollable events in supervisory control theory [11]. Assumption 2.6(ii) is the adversarial model under which the theorem is stated: the environment is assumed capable of interleaving at the worst moment. Assumption 2.6(iii) is satisfied by every decision function that operates on observable state; a function with oracular access to future states would not correspond to any implementable system. The three assumptions hold collectively in all multi-agent systems where agents share mutable state with concurrent external processes.

Example 2.7 (Running Example)**.** An agent manages file operations on a shared resource. The state s encodes the set of locked files $locked(s) \subseteq \mathcal{F}$ and the current quota usage $quota(s) \in \mathbb{N}$. Action $write(f)$ is admissible in s iff $f \notin locked(s)$ and $quota(s) < quota_{\max}$. Environment action $lock(f)$ transitions any state s to state $s^* = s[locked \mapsto locked(s) \cup \{f\}]$, making $write(f)$ inadmissible. Assumption 2.5 is satisfied.

3 The Decision Boundary Model

3.1 Decision and Transition Functions

Definition 3.1 (Decision Domain)**.** Let $\mathcal{D} = \{\textsc{Allow}, \textsc{Refuse}, \textsc{Escalate}\}$. We call elements of $\mathcal{D}$ *dispositions*.

Definition 3.2 (Decision Function)**.** A *decision function* is a mapping $D : S \times A \to \mathcal{D}$ that assigns a disposition to each (state, action) pair. We say D is *consistent with* Adm if the following hold for all $(s, a) \in S \times A$:

(i) $D(s, a) = \textsc{Allow} \Rightarrow \mathrm{Adm}(s, a)$;

(ii) $D(s, a) = \textsc{Refuse} \Rightarrow \neg\mathrm{Adm}(s, a)$;

(iii) $\mathrm{Adm}(s, a) \Rightarrow D(s, a) \neq \textsc{Refuse}$ (admissible states are never refused without escalation);

(iv) $\neg\mathrm{Adm}(s, a) \Rightarrow D(s, a) \neq \textsc{Allow}$ (inadmissible states are never allowed).

Conditions (i)–(iv) together imply: $D(s, a) = \textsc{Allow} \Leftrightarrow \mathrm{Adm}(s, a)$ whenever $D(s, a) \neq \textsc{Escalate}$. The Escalate disposition is reserved for cases where admissibility is indeterminate and requires external resolution.

Definition 3.3 (Transition Function)**.** A *transition function* is a mapping $T : S \times A \to S$ such that $T(s, a)$ gives the resulting state when action a is applied in state s.

Remark 3.4 (The role of F versus composition of D and T)*.* One might observe that the type signature $F : S \times A \to \mathcal{D} \times S$ is mathematically identical to the pointwise pair (D, T): given (s, a), compute $D(s, a)$ for the disposition and $T(s, a)$ for the resulting state. Any system—including a split one—can define such a function as a mathematical object. The contribution of the Atomic Decision Boundary (Definition 3.5) is not in the signature but in the *indivisibility axiom*: the computation of (d, s') must correspond to a *single arc* in the LTS, exposing no intermediate state to environment actions. In a split system, the same pair (d, s') is eventually produced, but the path from s to s' passes through the intermediate state s_D (introduced formally in Definition 3.7 below, §3.3) where environment actions can interleave. The structural difference between the two classes is in the topology of the LTS—specifically, whether there exists an intermediate state—not in the domain or codomain of the function.

3.2 Atomic Decision Boundary

Definition 3.5 (Atomic Decision Boundary)**.** A system satisfies an *atomic decision boundary* if there exists a function

$$F : S \times A \to \mathcal{D} \times S$$

such that for every $(s, a) \in S \times A$:

$$F(s, a) = (d,\ \hat{s}') \tag{1}$$

$$d = D(s, a) \tag{2}$$

$$\hat{s}' = (T(s, a),\ P) \quad \text{if } d = \textsc{Allow} \tag{3}$$

$$\hat{s}' = (s,\ P) \quad \text{if } d = \textsc{Refuse} \tag{4}$$

$$\hat{s}' = (s,\ P \cup \{(s, a)\}) \quad \text{if } d = \textsc{Escalate} \tag{5}$$

where $\hat{s}' \in \hat{S} = S \times \text{Req}$ is the post-transition state in the escalation-extended state space (defined formally in Definition 3.8, §3.4). For $d \in \{\textsc{Allow}, \textsc{Refuse}\}$, the pending-request component P is unchanged by the transition; the state reduces to S in those cases. The computation of d and $\hat{s}'$ corresponds to a *single indivisible transition* in the LTS: $s \xrightarrow{F(s,a)} \hat{s}'$ with no intermediate state between the evaluation of d and the determination of $\hat{s}'$.

The indivisibility condition in Definition 3.5 is a model-level axiom: it asserts that no environment action $e \in A_{\text{env}}$ can be interleaved between the evaluation of $D(s, a)$ and the firing of $T(s, a)$. In terms of the LTS, the $F(s, a)$ transition is a single arc; it does not decompose into two separate transitions with any state in between.

Remark 3.6 (Implementation independence). Atomicity in Definition 3.5 is a structural property of the system architecture, not of any particular implementation. Whether it is realized via a database transaction, a monitor lock, a hardware instruction, or a protocol contract is an implementation matter. The definition abstracts over all of these and asserts only that the LTS representation of the system does not expose an intermediate state.

3.3 Split Evaluation Systems

Definition 3.7 (Split Evaluation System)**.** A system is a *split evaluation system* if the decision function D and the transition function T are evaluated as distinct LTS transitions, potentially separated by environment actions. Formally, the execution of agent action a from state s decomposes into:

(1) A *decision transition*: $s \xrightarrow{dec(a)} s_D$, where s_D encodes the recorded disposition $d = D(s, a)$ alongside the current state.

(2) An optional sequence of environment transitions: $s_D \xrightarrow{e_1} \cdots \xrightarrow{e_k} s^*$, for $k \geq 0$.

(3) An *execution transition*: $s^* \xrightarrow{exec(a)} T(s^*, a)$, contingent on $d = \textsc{Allow}$.

The states s (evaluation state) and s^* (execution state) may differ whenever $k \geq 1$.

The structural distinction between atomic and split systems is not a matter of policy expressiveness, state availability, or evaluation latency. It is a matter of whether the LTS exposes an intermediate state between decision and execution. In atomic systems it does not; in split systems it does.

3.4 Escalation Semantics

The Escalate disposition is structurally distinct from Refuse: rather than terminating the request, it suspends the transition pending external review. This outcome has no counterpart in classical TOCTOU analyses, which operate in binary (permit/deny) models. We give it a precise semantics.

Definition 3.8 (Escalation State)**.** Let $\text{Req} = \mathcal{P}_{\text{fin}}(S \times A)$ denote the set of finite sets of *pending requests*. The *escalation-extended state space* is $\hat{S} = S \times \text{Req}$. A state $\hat{s} = (s, P) \in \hat{S}$ encodes the current system state s together with the set P of pending (state, action) pairs awaiting supervisor resolution.

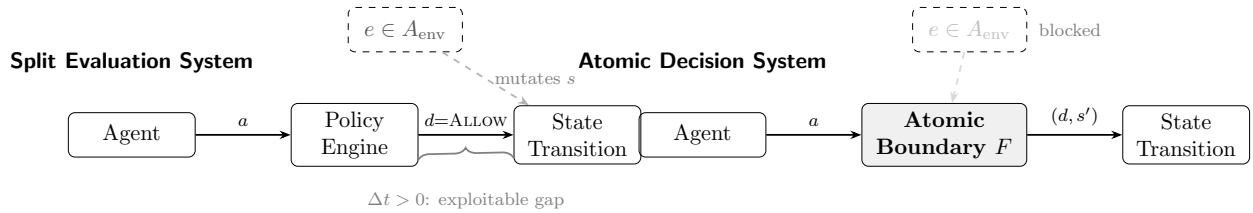

Figure 1: Architectural contrast between a split evaluation system (left) and an atomic decision system (right). In the split system, environment actions $e \in A_{\text{env}}$ may interleave between the policy evaluation and the state transition, producing the exploitable gap $\Delta t > 0$ proved in Theorem 5.1. In the atomic system, F determines both the decision and the resulting state as a single indivisible LTS step; no environment action can interleave within it.

Definition 3.9 (Escalation Transition)**.** When $F(s, a) = (\textsc{Escalate}, \hat{s}')$, the system transitions to

$$\hat{s}' = \big(s,\ P \cup \{(s, a)\}\big),$$

recording the pending request (s, a) with the originating state s, while leaving the system state unchanged. No transition of T fires.

Definition 3.10 (Supervisor Resolution Function)**.** The *resolution function* resolve : $\hat{S} \times (S \times A) \times \{\textsc{Allow}, \textsc{Refuse}\} \to \hat{S}$ is defined, for $(s_{\text{orig}}, a) \in P$ (precondition: the request being resolved must be in the pending set), by:

$$\text{resolve}\big((s_t, P),\ (s_{\text{orig}}, a),\ \textsc{Allow}\big) = \big(T(s_t, a),\ P \setminus \{(s_{\text{orig}}, a)\}\big) \tag{6}$$

$$\text{resolve}\big((s_t, P),\ (s_{\text{orig}}, a),\ \textsc{Refuse}\big) = \big(s_t,\ P \setminus \{(s_{\text{orig}}, a)\}\big) \tag{7}$$

where s_t is the *current* system state at resolution time, which may differ from s_{orig}. The function is undefined when $(s_{\text{orig}}, a) \notin P$; resolution of a request not in the pending set is a protocol error.

Two observations are immediate. First, when ALLOW is issued at resolution, T fires in state s_t, not s_{orig}. The supervisor therefore cannot rely on the admissibility evaluation performed in s_{orig}; it must re-evaluate $\text{Adm}(s_t, a)$. Second, the resolution function itself is a decision point: it evaluates a disposition (ALLOW or REFUSE) and potentially fires a transition ($T(s_t, a)$). This is precisely the structure of a decision boundary.

Remark 3.11 (The supervisor as a second decision system)*.* The ESCALATE outcome introduces a second decision point operated by an external supervisor, which raises a natural question: is this the same system, or a different one? The model is agnostic. The supervisor may be a human reviewer, an automated policy agent, or a recursive invocation of the same governance pipeline. What Corollary 5.6 (to follow) establishes is independent of this: *wherever* the supervisor sits, its resolution function resolve constitutes a decision boundary of its own, and the same structural argument that applies to the primary boundary applies there. The escalation model does not solve the atomicity problem by deferral; it makes explicit that the problem *cannot* be avoided by deferral, only transferred.

4 The Failure of Split Systems

Before stating the main theorem, we illustrate the failure mode concretely.

Example 4.1 (Admissibility Violation in a Split System)**.** Return to Example 2.7. The following three-step trace witnesses the failure (steps correspond to those in the proof of Theorem 5.1):

(1) The system is in state s with $f \notin \mathit{locked}(s)$. The agent requests $\mathit{write}(f)$; the split system evaluates $D(s, \mathit{write}(f)) = \textsc{Allow}$ and records the decision.

(2) Before execution fires, another agent issues $\mathit{lock}(f)$, an environment action that transitions $s \to s^*$ with $f \in \mathit{locked}(s^*)$.

(3) The original decision ($\textsc{Allow}$) is already recorded. The execution transition $\mathit{exec}(\mathit{write}(f))$ fires in s^*, applying $T(s^*, \mathit{write}(f))$.

At the moment of execution, $\mathrm{Adm}(s^*, \mathit{write}(f)) = \mathsf{false}$: the file is locked. The split system has executed an inadmissible transition.

In contrast, an atomic system would evaluate $F(s^*, \mathit{write}(f))$ at the moment of the transition and obtain $(\textsc{Refuse}, s^*)$, preventing the inadmissible execution.

The failure in Example 4.1 cannot be remedied by:

- **Richer policies.** The decision function D already produces the correct result for every state it is given. The problem is that it is given s, not s^*.
- **External state.** Adding an external store (e.g., a shared lock registry read by D) reduces the window in which the gap can be exploited but does not reduce it to zero. The store must be read before T fires; between that read and the firing of T, the store can be updated.
- **Re-evaluation before execution.** If the system re-evaluates D immediately before firing T, the re-evaluation and the execution are themselves two separate transitions—which is exactly the structure of a split system. Unless the re-evaluation and the execution are jointly atomic, the gap is merely shifted, not closed.

5 Main Results

5.1 Non-Equivalence Theorem

We now prove the central result. Throughout this section, *equivalence* between two systems is taken with respect to *admissibility preservation* (Definition 2.4): two systems are equivalent if and only if they agree on which execution traces preserve admissibility under all environments satisfying Assumptions 2.5 and 2.6. The theorem is stated for the general case; the proof is constructive via the execution trace in Example 4.1.

Theorem 5.1 (Non-Equivalence of Split and Atomic Systems)**.** *Let M_{split} be any split evaluation system and let M_{atom} be any atomic decision boundary system, both defined over the same state space S, action set A, environment actions A_{env}, admissibility predicate* Adm*, and transition function T. Under Assumptions 2.5 and 2.6, there exists an execution trace $\sigma^* \in \mathrm{Traces}(M_{\mathrm{split}})$ such that:*

(i) σ^* contains a transition $s^* \xrightarrow{\mathit{exec}(a)} T(s^*, a)$ with $\mathrm{Adm}(s^*, a) = \mathsf{false}$.

(ii) No corresponding trace exists in $\mathrm{Traces}(M_{\mathrm{atom}})$ that commits $T(\cdot, a)$ in a state where $\mathrm{Adm}(\cdot, a) = \mathsf{false}$.

Consequently, M_{split} does not preserve admissibility, while M_{atom} does.

Proof. By Assumption 2.5(i), there exist $s \in S$ and $a \in A$ with $\mathrm{Adm}(s, a) = \mathsf{true}$. By Assumption 2.5(ii), there exists $e \in A_{\mathrm{env}}$ and $s^* \in S$ with $s \xrightarrow{e} s^*$ and $\mathrm{Adm}(s^*, a) = \mathsf{false}$.

Constructing σ^*. Consider the following trace in M_{split}:

$$\sigma^* : \quad s \xrightarrow{dec(a)} s_D \xrightarrow{e} s^* \xrightarrow{exec(a)} T(s^*, a).$$

1. The decision transition $dec(a)$ fires in state s. By Assumption 2.5(i), $D(s, a) = \textsc{Allow}$. The system records $\textsc{Allow}$ and transitions to s_D (encoding the recorded decision alongside the current state; the underlying system state is still s).

2. The environment transition e fires, producing s^* with $\text{Adm}(s^*, a) = \mathsf{false}$.

3. The execution transition $exec(a)$ fires in s^*. In M_{split}, execution is triggered by the recorded disposition $\textsc{Allow}$, which was evaluated in s. The split system applies $T(s^*, a)$, yielding the inadmissible transition.

By construction, $\sigma^* \in \text{Traces}(M_{\text{split}})$ (all three transitions are valid in a split system), and the transition at step (3) is inadmissible in s^*. This establishes (i).

Atomic system behavior. In M_{atom}, the agent action a corresponds to a single indivisible LTS transition $F(s', a)$ from whatever state s' holds at the moment the agent acts.

If the agent acts *before* the environment transition e fires (system in state s): $F(s, a) = (\textsc{Allow}, T(s, a))$. The transition completes atomically; e cannot interleave *within* the single LTS step. The resulting state is $T(s, a)$, produced from s where $\text{Adm}(s, a)$ holds.

If the agent acts *after* e fires (system in state s^*): $D(s^*, a) \in \{\textsc{Refuse}, \textsc{Escalate}\}$, since $\text{Adm}(s^*, a) = \mathsf{false}$ and D is consistent with Adm (condition (iv): $\neg\text{Adm} \Rightarrow D \neq \textsc{Allow}$). In either case, no transition of T fires: $\textsc{Refuse}$ keeps the state unchanged, and $\textsc{Escalate}$ records the pending request without firing T (Definition 3.9).

In neither case does M_{atom} commit $T(\cdot, a)$ in a state where admissibility fails. This establishes (ii).

Conclusion. M_{split} produces an inadmissible transition under σ^*; M_{atom} does not. Therefore M_{split} does not satisfy Definition 2.4 (admissibility preservation) under all execution traces. No construction applied to M_{split} can restore admissibility preservation without introducing an atomic boundary: any mechanism that prevents the violation in σ^* must either (a) block environment actions from interleaving between $dec(a)$ and $exec(a)$—which is precisely Definition 3.5—or (b) re-evaluate D at execution time jointly with T as a single indivisible step—which is again Definition 3.5. In both cases, the fix is the atomic boundary condition itself. □

Remark 5.2 (Scope and boundary conditions of the impossibility). Theorem 5.1 is a conditional result: it holds under Assumptions 2.5 and 2.6. In a *sequential* system where no concurrent environment actions occur ($k = 0$ always in Definition 3.7), the trace σ^* is unreachable and the gap cannot be exploited. Similarly, if mutual exclusion is enforced at the operating-system or runtime level over the *entire* interval between $dec(a)$ and $exec(a)$, the split system becomes effectively atomic within that exclusion scope—but this is precisely an implementation of the atomic boundary (see Remark 3.6).

Two common objections deserve direct answers. *First*: "What about locks, transactions, CAS, or linearizable storage?" These are valid implementation mechanisms for the atomic boundary; they are not alternatives to it. A system that wraps both the decision call and the execution call inside a single database transaction, or acquires a lock before $dec(a)$ and releases it only after $exec(a)$, satisfies Definition 3.5 by construction—because no environment action can fire between the two.

Second: "Isn't this just data atomicity?" No. Data atomicity (2PC, MVCC) ensures that a data record is fully written or not written at all. Admissibility atomicity ensures that the *policy check* and the *data write* are evaluated against the same system-state snapshot. A 2PC-coordinated write preceded by an OPA policy call is still a split system in our model (§7.1).

5.2 Corollaries

Corollary 5.3 (Impossibility of Execution-Time Guarantees in Split Systems)**.** *A split evaluation system cannot guarantee admissibility at execution time under all execution traces.*

Proof. Immediate from Theorem 5.1: the trace σ^* is a constructive witness to a violation. Since σ^* is a valid trace of any split system satisfying Assumption 2.5, no such system can claim universal admissibility preservation. □

Corollary 5.4 (External State Does Not Restore Atomicity)**.** *Augmenting a split system with access to external state does not restore equivalence with an atomic system unless evaluation and the resulting state transition are jointly enforced as a single indivisible step.*

Proof. External state enriches the domain of the decision function D: let $D' : S \times A \times \mathcal{E} \to \mathcal{D}$ where $\mathcal{E}$ is the external state. The augmented system still decomposes into a decision transition (invoking D' with the external state read at that moment) and a subsequent execution transition. The environment action e in σ^* may act on the external state or the system state between these transitions; the structural gap is unchanged. Formally, $D'(s, a, \epsilon)$ with $\epsilon = \epsilon_s$ (the external state corresponding to s) may return Allow, while by the time T fires the external state has evolved to ϵ_{s^*} reflecting the lock acquired by e. The gap in σ^* is not closed. Atomic coupling of D' and T is required. □

Corollary 5.5 (Admissibility is a Property of Execution)**.** *Admissibility cannot be reduced to a property of evaluation alone; it is a property of the state at the moment the transition is committed.*

Proof. In the trace σ^*, the same action a is admissible in state s (evaluation state) and inadmissible in state s^* (execution state). Since execution commits the transition in s^*, admissibility must be determined with respect to s^*. Any governance guarantee that is computed only at s is therefore insufficient. □

5.3 The Escalation Closure Requirement

Corollary 5.6 (Escalation Closure Requirement)**.** *An* Escalate *outcome preserves the atomic decision boundary guarantee if and only if the supervisor resolution function* resolve *is itself atomic: the evaluation of* $\mathrm{Adm}(s_t, a)$ *at resolution time and the application of* $T(s_t, a)$ *must be jointly determined as a single indivisible step.*

Proof. Let the system issue Escalate for action a in state s, recording the pending request (s, a) per Definition 3.9.

(Necessity.) Suppose resolve is split: the supervisor evaluates $D(s_t, a)$ in state s_t, and then fires T separately.

We verify that Assumptions 2.5 and 2.6 hold for the resolution sub-process operating on $\hat{S} = S \times$ Req. Assumption 2.5(i) holds because (s_t, a) has $\mathrm{Adm}(s_t, a) = \mathsf{true}$ (otherwise the supervisor would not issue Allow) and $D(s_t, a) =$ Allow (this is the split supervisor's decision). Assumption 2.5(ii) holds because the system state s_t (the first component of $\hat{S}$) can be modified by environment actions between the supervisor's decision transition and the execution of T; the same Assumption 2.6 that

applies to the primary system applies to the projected S-component of $\hat{S}$. Assumption 2.6(i)–(iii) carry over verbatim to the projected S-component: environment actions are uncontrollable, may interleave between the supervisor's evaluation and the firing of T, and the supervisor observes only the current state s_t at the time of evaluation.

By Theorem 5.1 applied to this resolution sub-process, there exists a trace in which the supervisor's ALLOW is computed in s_t but T fires in s^{**} with $\mathrm{Adm}(s^{**}, a) = \mathsf{false}$. Thus, resolve admits an inadmissible transition.

(Sufficiency.) Suppose resolve is atomic: the evaluation $D(s_t, a)$ and the application $T(s_t, a)$ are a single indivisible step in the LTS. Then by Definition 3.5 applied to resolve, the resolution satisfies the atomic decision boundary condition, and Theorem 5.1 guarantees no inadmissible transition at resolution. □

Corollary 5.6 has an important practical consequence: the ESCALATE outcome does not relax the atomicity obligation; it *transfers* it from the primary enforcement point to the supervisor. A chain of escalations each requiring their own atomic resolution corresponds precisely to the notion of a supervisory hierarchy in discrete event systems [11].

6 Mapping to Existing Systems

Table 1 classifies common governance mechanisms according to the distinction established in §3. The classification is structural and does not reflect comparisons of expressiveness or performance.

Table 1: Structural classification of existing governance mechanisms.

System	**Class**	**Basis for classification**
RBAC [12]	Split	Permission check and execution are distinct operations
ABAC [7]	Split	Attribute evaluation external to transition; attributes may change
OPA [10]	Split	Policy evaluation external to state transition
Cedar [3]	Split	Authorization decision external to the enforcing runtime
OPA + Redis	Split	External state enriches D; structural gap remains (Cor. 5.4)
AWS IAM	Split	IAM evaluation precedes API call execution; no shared-snapshot guarantee
Kubernetes Admission	Partially atomic	Webhook admits before object creation, but cross-resource state (e.g., a `ResourceQuota` modified concurrently) is not jointly atomic with the admission decision (Remark 6.1)
ACP [4]	Atomic	Decision and state mutation jointly enforced via single-use Execution Token

Role-Based Access Control (RBAC). RBAC assigns permissions to roles and roles to principals. At access time, the system evaluates whether the requesting principal holds a role that permits the requested operation. The evaluation produces a disposition; the operation is then executed by the application. The RBAC system has no mechanism to ensure that the role assignment—or the system state on which the permission check depends—remains unchanged between evaluation and execution. RBAC implements:

$$d = D_{\mathrm{RBAC}}(s, a), \quad s' = T(s, a),$$

as separate operations. RBAC is a split evaluation system.

Concrete trace. In state s, an agent holds the EDITOR role and requests $\mathit{write}(r)$. The RBAC check returns $d = \textsc{Allow}$. Before the write executes, an administrator revokes the role via environment action e, transitioning to s^* where the agent is no longer an EDITOR and $\mathrm{Adm}(s^*, \mathit{write}(r)) = \mathsf{false}$. The recorded ALLOW nonetheless triggers execution of $T(s^*, \mathit{write}(r))$. This is precisely the trace σ^* of Theorem 5.1.

Open Policy Agent (OPA). OPA evaluates policies expressed in Rego against an input document. The evaluation produces a decision; execution is performed externally by the calling system. In its standard deployment model:

$$d = D_{\mathrm{OPA}}(s, a),$$

with $T(s, a)$ executed outside the policy engine. Even when OPA is augmented with external data injected at query time (e.g., via the bundle API or a data source), the execution transition remains external. By Corollary 5.4, this augmentation does not restore atomicity. OPA is a split evaluation system.

Concrete trace. An OPA bundle is evaluated against input document encoding state s (e.g., current API quota usage $= q < q_{\max}$) and returns $d = \textsc{Allow}$. Between the policy call and the application's write execution, a concurrent request exhausts the remaining quota, producing s^* with $\mathrm{Adm}(s^*, \mathit{write}) = \mathsf{false}$. The application, holding the recorded ALLOW, proceeds. Injecting a real-time Redis data source into OPA narrows the window but does not close it: the Redis read is itself a distinct LTS transition exposed to interleaving between $\mathit{dec}(a)$ and $\mathit{exec}(a)$ (Corollary 5.4).

ACP (Agent Control Protocol). ACP implements admission control as a stateful runtime mechanism in which the decision and state mutation are jointly enforced at the execution boundary [4]. Each agent action must obtain an Execution Token—a single-use cryptographic proof of admission—before the action is permitted to proceed. The token is issued and consumed atomically relative to the ledger state, ensuring that the system state against which admissibility is evaluated is the same state against which the transition commits.

ACP further implements the ESCALATE outcome as a pending-review state tracked in the Audit Ledger (ACP-LEDGER-1.3), where subsequent resolution passes through the same admission pipeline. By Corollary 5.6, this satisfies the escalation closure requirement.

Formally:

$$F_{\mathrm{ACP}}(s, a) = (d, s'),$$

where d and s' are determined by a single evaluate-then-mutate execution contract [4]. ACP is an atomic decision boundary system.

AWS IAM. AWS IAM evaluates authorization requests against a set of policies attached to principals, resources, and conditions. An IAM authorization call (e.g., to the IAM policy simulator or to the underlying STS service) produces a decision of ALLOW or REFUSE. The resulting API call is executed by the service (e.g., S3, EC2) independently of the IAM evaluation context. No shared-snapshot guarantee exists between the IAM policy evaluation context and the service's execution context: condition keys (e.g., `aws:CurrentTime`, `ec2:ResourceTag`) are evaluated at IAM call time, while the resource state at execution time may differ.

Concrete trace. An agent requests an S3 write. IAM evaluates the policy in state s where the bucket's ACL permits writes from the agent's role, returning $d = \textsc{Allow}$. Before the write reaches S3, a concurrent process changes the bucket policy (environment action e), producing s^* where the write is inadmissible. The S3 service executes $T(s^*, \mathit{write})$ regardless, since it received the IAM authorization token produced in s. This is the trace σ^* of Theorem 5.1. AWS IAM is a split evaluation system.

Cedar. Cedar is a policy language and evaluation engine designed for authorization decisions in service-oriented architectures [3]. Like OPA, Cedar evaluates policies against entities (principal, action, resource, context) and returns a decision. The enforcement of that decision—the actual execution of the requested operation—occurs outside Cedar, in the calling application or service. Cedar provides no mechanism to jointly enforce its authorization decision with the subsequent state transition; the two remain separate LTS transitions. By Corollary 5.4, augmenting Cedar with an entity store does not restore atomicity: the entity store is read at evaluation time, and the application may mutate relevant entity attributes between Cedar's decision and the action's execution. Cedar is a split evaluation system.

Remark 6.1 (Partially atomic systems). Table 1 uses the term *partially atomic* for Kubernetes admission webhooks. This denotes a system that satisfies the atomic decision boundary condition (Definition 3.5) for a restricted subset of state—specifically, single-object lifecycle operations—but fails atomicity when the relevant admissibility predicate Adm depends on shared or cross-resource state that is not included in the joint evaluation-and-execution snapshot. Formally: let S decompose as $S = S_{\text{local}} \times S_{\text{global}}$. If the system is atomic with respect to S_{local} but split with respect to S_{global}, it is partially atomic. When Adm depends only on S_{local}, the system provides full guarantees; when Adm depends on S_{global}, Theorem 5.1 applies to the S_{global} component.

Kubernetes Admission (Partially Atomic). Kubernetes admission webhooks intercept API server requests before the object is written to etcd, providing a form of pre-execution evaluation. Within a single object's lifecycle, the webhook decision and the subsequent write to etcd are reasonably coupled. However, Kubernetes does not enforce atomicity across resources: consider an agent Pod requesting a batch job that is admissible given the current `ResourceQuota` state. Between the webhook's admission decision and the Pod's binding to a node, a concurrent namespace operation updates the `ResourceQuota` to a value that would have caused refusal. The transition fires in an inadmissible cross-resource state. In terms of our model, the state s relevant to Adm encodes cluster-wide resource state; the admission decision is evaluated in s but the execution commits in a state s^* that the webhook did not observe. The system is partially atomic (per-object) but split with respect to the full admissibility predicate over shared cluster state.

7 Discussion

7.1 Relationship to Distributed Transactions

Two-phase commit (2PC) and multiversion concurrency control (MVCC) achieve atomicity for *data transitions*: they ensure that a database record is either fully updated or not updated at all. This is a different plane from the admissibility atomicity we define.

Admissibility is a predicate over the system state at transition time; its enforcement requires that the policy check and the transition commit occur in the same snapshot of that state. 2PC coordinates multiple data stores to commit together; it does not ensure that the policy check preceding the commit was evaluated in the same snapshot. A system can run an OPA policy check followed by a 2PC-coordinated database update and still be a split system in our model: the OPA call and the 2PC commit are separate operations with an exploitable window between them.

An atomic decision boundary requires that the policy evaluation and the state commit are *the same operation*—not merely that they are coordinated.

Relationship to linearizability. Linearizability [8] is the standard correctness criterion for concurrent objects: an operation appears to take effect instantaneously at some point between its invocation and its response. This is a property of *data operations* on shared objects. The atomic decision boundary operates at a distinct level of abstraction: it is a property of *governance decisions* over state transitions.

A linearizable data store guarantees that each read or write appears atomic in isolation; it does not guarantee that a policy check and the write it governs were evaluated against the same state snapshot. A system that runs a policy evaluation followed by a linearizable write is still a split system in our model: the policy call and the write are separate operations with an exploitable window between them.

These are complementary guarantees, not competing ones. A system achieves the atomic decision boundary by implementing the policy check and the resulting state transition as a *single* linearizable operation— precisely collapsing the gap. Linearizability specifies what it means for a single operation to be atomic; the atomic decision boundary specifies *which operations must be made atomic together*.

7.2 What Atomicity Does Not Prescribe

Our results characterize a *necessary* structural condition for admissibility guarantees. They do not:

- prescribe a specific implementation of the atomic boundary (hardware CAS, database transaction, protocol contract, or otherwise);
- claim that atomic systems are immune to all governance failures— behavioral drift operating below the enforcement threshold is addressed separately [6];
- argue that split systems have no valid use. Split systems can provide probabilistic assurance, reduce latency, or serve in contexts where admissibility is defined conservatively. The theorem establishes only that they cannot provide the universal guarantee.

7.3 Concurrency Models

The trace σ^* in Theorem 5.1 assumes an adversarial environment: the environment action e is assumed to be able to fire between $dec(a)$ and $exec(a)$. In a stochastic model, e fires with some

probability $p > 0$; the guarantee degrades from universal to probabilistic. In a sequential model where no concurrency is possible, the gap does not arise. The relevance of Theorem 5.1 is therefore proportional to the degree of concurrency in the deployment environment. For modern multi-agent systems operating over shared state, $p > 0$ is the realistic assumption.

7.4 Relationship to Formal Verification

TLA+ and other model-checking frameworks verify safety and liveness properties of system models. In a TLA+ model of a split system, the gap in σ^* can be made explicit: if environment actions are part of the model, the checker will find traces that violate admissibility invariants. This paper provides the structural characterization that motivates where to look and why no refinement of the split architecture can eliminate the counterexample trace. The ACP specification [4] is model-checked with TLC and verifies safety invariants that hold precisely because ACP enforces an atomic boundary.

8 Related Work

Time-of-Check/Time-of-Use (TOCTOU). The TOCTOU pattern—in which a security check and the operation it governs are separated in time—has been studied extensively in operating systems security [2, 14]. The classic form involves a file system race: a process checks file permissions and then opens the file; an attacker replaces the file between check and open. Our model generalizes TOCTOU from OS primitives to governance systems, with three key extensions.

First, we operate at the level of *state transitions* rather than file handles: the relevant state is the full system state encoding all properties on which admissibility depends, not merely a permission bit. Second, we add the Escalate outcome, which TOCTOU analyses do not consider. Classical TOCTOU mitigation strategies (e.g., using `openat` or atomic rename) work by removing the gap; our framework shows that the same principle applies to governance systems and that the gap cannot be closed by re-evaluation alone. Third, we provide a positive characterization of systems that are immune to the gap (atomic systems), not only a characterization of vulnerable systems.

Supervisory Control of Discrete Event Systems. Ramadge and Wonham [11] introduced the formal theory of supervisory control, in which a supervisor enables or disables events in a plant modeled as a DES. Their framework distinguishes controllable events (which the supervisor can prevent) from uncontrollable events (which it cannot). Our framework is related: agent actions correspond to controllable events; environment actions correspond to uncontrollable ones. The Escalate outcome corresponds to the supervisor suspending a controllable event pending further observation—a concept formalized in the DES literature as *conditional enabling* [11]. Corollary 5.6 is the governance-system analogue of the requirement that supervisory decisions be enforced at the moment of event firing.

Access Control. The access matrix model of Harrison, Ruzzo, and Ullman [7] is the classical foundation of access control theory. RBAC [12] and attribute-based access control (ABAC) elaborate on it. These models define what operations principals are permitted to perform; they do not model the temporal relationship between the permission check and the operation execution. Our work complements access control theory by characterizing when that temporal relationship matters for correctness.

Reference Monitors. Anderson [1] defined the reference monitor as a component that must be invoked on every access, must be tamper-proof, and must be small enough to be verified. Our atomic decision boundary captures the first and third properties at a formal level: the boundary is invoked at the moment of transition (complete mediation) and its definition is minimal (a single function F). The ACP protocol [4] is explicitly designed as a decentralized reference monitor; the present paper provides the formal basis for why complete mediation requires atomicity.

Runtime Enforcement and Security Automata. Schneider [13] characterized which security properties are enforceable by inline reference monitors modeled as security automata: a property is enforceable if and only if its violations are detectable in finite traces (i.e., it is a safety property). The admissibility preservation requirement of Definition 2.4 is a safety property in precisely this sense—each violation is witnessed by the finite trace σ^*. Schneider's framework assumes the monitor can observe and halt executions; our work identifies the structural condition under which that halt can be guaranteed *atomically with respect to the state transition being governed*. A monitor that halts asynchronously with the transition is still a split system in our model.

Ligatti, Bauer, and Walker [9] extended security automata to *edit automata*, which can suppress or insert events to enforce a broader class of policies. The ESCALATE outcome in our model corresponds structurally to suppression: the transition is prevented pending external resolution. The Escalation Closure Requirement (Corollary 5.6) is the governance-layer analogue of the requirement that edit operations themselves be applied consistently with the current state of the running system—a condition edit automata assume implicitly but do not formalize at the level of decision boundaries.

Policy Engines. OPA [10] and Cedar [3] provide expressive policy languages for evaluating authorization decisions. These systems are powerful within their class (split evaluation) and our taxonomy does not imply that they are deficient—only that they cannot provide execution-time admissibility guarantees by construction. Hybrid architectures in which a policy engine provides the decision function D inside an atomic enforcement layer are consistent with our model; the atomicity requirement falls on the enclosing layer, not on the policy engine itself.

Temporal Logic and Model Checking. Safety properties in temporal logic (e.g., $\Box\neg\text{bad}$) can express admissibility requirements. Model checkers verify whether a system satisfies such properties. Our contribution is complementary: we identify which *system architectures* can satisfy admissibility properties universally (atomic) and which cannot (split), independent of any particular property specification.

9 Conclusion

We have introduced the *atomic decision boundary* as a structural requirement for governance systems that must enforce admissibility at execution time. Using a formal LTS model, we proved that split evaluation systems cannot provide this guarantee under all execution traces, and showed that no enrichment of the split architecture—whether through additional policies, external state, or re-evaluation—can close the structural gap without introducing atomicity.

The ESCALATE outcome extends classical binary decision models and captures supervisor-mediated governance. We proved that escalation transfers, rather than removes, the atomicity obligation: any system that resolves an ESCALATE must itself be atomic at the point of resolution.

The core result is direct: *admissibility is not a property of evaluation—it is a property of execution.* Under Assumptions 2.5 and 2.6, no system that separates evaluation from execution can guarantee admissibility at execution time; only a system that couples them as a single indivisible step at the decision boundary can provide this guarantee.

This characterization serves as the formal foundation for the subsequent papers in this series of four. The Agent Control Protocol (ACP) [4] (Paper 1) instantiates F as a concrete atomic boundary: each agent action acquires a single-use Execution Token that is issued and consumed in the same ledger operation, ensuring that the state against which admissibility is evaluated is precisely the state against which the transition commits. The Invariant Measurement Layer [6] (Paper 2) operates *above* the atomic enforcement layer—taking atomicity as given—and detects behavioral drift that accumulates across sequences of individually admissible actions, remaining invisible to per-action enforcement signals. Paper 3 [5] extends the framework to the fair allocation of governance decisions across agents competing under shared resource constraints. All three rest on the structural guarantee established here: that the boundary they operate relative to is, in fact, atomic.

References

[1] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, USAF Electronic Systems Division, 1972.

[2] Matt Bishop and David Bailey. A critical analysis of vulnerability taxonomies. Technical Report CSE-96-11, University of California at Davis, 1996.

[3] John Cutler et al. Cedar: A new policy language. In *USENIX Security Symposium*, 2023.

[4] Marcelo Fernandez. Agent control protocol: Admission control for agent actions. `https://arxiv.org/abs/2603.18829`, 2026. arXiv:2603.18829 [cs.CR], DOI:10.5281/zenodo.19561638.

[5] Marcelo Fernandez. Fair atomic governance: Allocating decision boundaries under shared resource constraints in multi-agent systems. Manuscript in preparation, 2026.

[6] Marcelo Fernandez. From admission to invariants: Measuring deviation in delegated agent systems. Manuscript in preparation, 2026.

[7] Michael A. Harrison, Walter L. Ruzzo, and Jeffrey D. Ullman. Protection in operating systems. *Communications of the ACM*, 19(8):461–471, 1976.

[8] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

[9] Jay Ligatti, Lujo Bauer, and David Walker. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security*, 4(1–2):2–16, 2005.

[10] Open Policy Agent Contributors. Open policy agent. `https://www.openpolicyagent.org`, 2026. Accessed April 2026.

[11] Peter J. Ramadge and W. Murray Wonham. Supervisory control of a class of discrete event processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987.

[12] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[13] Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

[14] Dan Tsafrir, Tomer Hertz, David Wagner, and Dilma Da Silva. Portably solving file TOCTTOU races with hardness amplification. In *USENIX Conference on File and Storage Technologies (FAST)*, 2008.